

RÉPUBLIQUE DÉMOCRATIQUE DU CONGO
UNIVERSITÉ DE KINSHASA



FACULTÉ DES SCIENCES ET TECHNOLOGIES
Mention Mathématiques, Statistique et Informatique
B.P. 190 KINSHASA XI

Master 1

TRAVAIL PRATIQUE DE SYSTÈME MULTI-AGENT

**Modélisation et simulation d'une station de recharge
pour véhicules électriques**

Projet 8 : Station de Recharge pour Véhicules Électriques

DUASENGE MAYANO Brummel
MAKWIZA MBALA Judicael
MFUSHI KAPALAY Stéphane

Professeur KASEREKA Selain
Doctorant MBAYANDJAMBE Alidor

Mai 2026

Résumé

Ce travail présente une modélisation multi-agent du fonctionnement d'une station de recharge pour véhicules électriques soumise à une demande aléatoire et à des ressources limitées. Le modèle retenu est un modèle à états discrets couplé à une file d'attente multi-serveurs hétérogène : les véhicules sont des agents capables d'arriver, d'attendre, de se connecter à une borne, d'être servis ou d'abandonner, tandis que les bornes alternent entre disponibilité, occupation, panne et maintenance. La simulation intègre des arrivées fluctuantes avec pic en soirée, des arrivées groupées, des pannes de bornes et une variation de la puissance délivrée liée au réseau. Les indicateurs étudiés sont le taux d'occupation moyen des bornes, la longueur moyenne de la file, le temps d'attente moyen, le taux d'abandon et le taux de satisfaction global. Le modèle est implémenté en GAML sur GAMA Platform, avec une configuration de 10 bornes lentes, 8 bornes accélérées et 5 bornes rapides.

1 Introduction

Le projet 8 porte sur la simulation du fonctionnement en temps réel d’une station de recharge pour véhicules électriques. Le problème étudié est celui d’un service soumis simultanément à une demande variable et à une capacité limitée : les véhicules n’arrivent pas de manière régulière, les bornes peuvent être occupées ou indisponibles, et les usagers ne tolèrent pas tous la même durée d’attente. Le sujet impose donc de représenter à la fois des ressources techniques, des comportements individuels et des indicateurs globaux de performance.

Une approche multi-agent est adaptée à ce type de système, car elle permet de représenter chaque entité active par ses états, ses décisions et ses interactions. Dans le cadre du cours de systèmes multi-agents, un agent est considéré comme une entité située dans un environnement, capable de percevoir certaines informations et d’agir de manière autonome. La station de recharge constitue ici un environnement dynamique et non déterministe : de nouveaux véhicules peuvent arriver, les bornes changent d’état, la puissance disponible varie, et les décisions locales produisent des effets collectifs visibles sur la file d’attente et la satisfaction.

Ce choix est cohérent avec les travaux scientifiques consacrés aux stations de recharge. Les modèles de files d’attente sont utilisés pour dimensionner les stations et estimer la qualité de service [4], tandis que les modèles stochastiques à arrivées variables permettent de représenter un flux entrant dépendant du temps [5]. Les simulations agent-based sont également employées pour étudier les temps d’attente, les politiques de coordination et l’utilisation des chargeurs [6, 7]. Notre modèle combine ces idées à l’échelle d’une station : agents individuels, états discrets, file d’attente et indicateurs de performance.

L’objectif de ce travail est triple. Premièrement, il s’agit de proposer un modèle multi-agent à états discrets décrivant les véhicules, les bornes et la file d’attente. Deuxièmement, il faut formaliser les principales équations de fonctionnement déduites du code GAML : arrivée des véhicules, durée de recharge, attente, abandon, occupation et satisfaction. Troisièmement, le modèle doit être simulé dans GAMA Platform afin d’analyser les sorties graphiques produites par l’expérience.

Le rapport est organisé de la manière suivante. La section 2 présente brièvement le cadre SMA et GAMA utilisé. La section 3 décrit le modèle des véhicules, des bornes et de la file. La section 4 donne la formulation mathématique issue du code. La section 5 précise les paramètres de simulation. La section 6 résume l’implémentation GAML. La section 7 présente les résultats obtenus et leur interprétation. Enfin, la conclusion synthétise les apports et limites du modèle.

2 Cadre SMA et GAMA

Le système étudié est composé de plusieurs agents qui interagissent autour d'une ressource limitée. Les véhicules sont des agents dynamiques : ils arrivent dans la station, évaluent la disponibilité des bornes, rejoignent éventuellement une file et peuvent abandonner si l'attente devient incompatible avec leur tolérance ou leur urgence. Les bornes sont des agents techniques : elles peuvent être libres, occupées, en panne ou en maintenance. La station joue le rôle d'agent de coordination, car elle conserve la file d'attente et attribue les bornes libres selon une politique de service.

Le modèle est réactif : les comportements sont principalement décrits par des règles exécutées à chaque pas de temps. Dans GAML, ces comportements sont codés par des **reflex**. Cette structure correspond à la logique des tutoriels GAMA utilisés dans le cours : les variables globales définissent le contexte, les **species** définissent les types d'agents, les **reflex** décrivent les comportements, et l'**experiment** regroupe les paramètres, les moniteurs et les graphiques.

GAMA Platform est retenue comme environnement de simulation, car elle fournit un langage de modélisation multi-agent, GAML, et des sorties graphiques directement exploitables, notamment les séries temporelles, histogrammes et diagrammes circulaires. Ces sorties sont utiles pour analyser simultanément les comportements microscopiques et les indicateurs macroscopiques du système.

3 Description du modèle

3.1 États des véhicules

Les véhicules ne sont pas modélisés comme une population continue, mais comme des agents individuels. Chaque véhicule possède un état discret. Un véhicule nouvellement créé est dans l'état **arriving**. S'il trouve une borne libre, il passe directement à l'état **charging**. Si aucune borne n'est libre, il rejoint l'état **waiting**. Pendant l'attente, il réévalue sa situation : si l'attente estimée dépasse sa tolérance, ou si son urgence est élevée après une attente prolongée, il passe à l'état **abandoned**. Lorsqu'une recharge se termine, le véhicule passe à l'état **served**, puis disparaît du modèle après comptabilisation des indicateurs.

3.2 États des bornes

Les bornes suivent quatre états. Une borne **libre** peut recevoir un véhicule et devenir **occupée**. Pendant la recharge, elle reste occupée jusqu'à ce que le véhicule atteigne son objectif de batterie ou que le temps de recharge restant devienne nul. Une borne libre ou occupée peut tomber en **panne** selon une probabilité par pas de temps. Si la panne survient pendant une recharge, le véhicule branché est replacé en tête de file. Après la durée de panne, la borne passe brièvement en **maintenance**, puis redevient disponible.

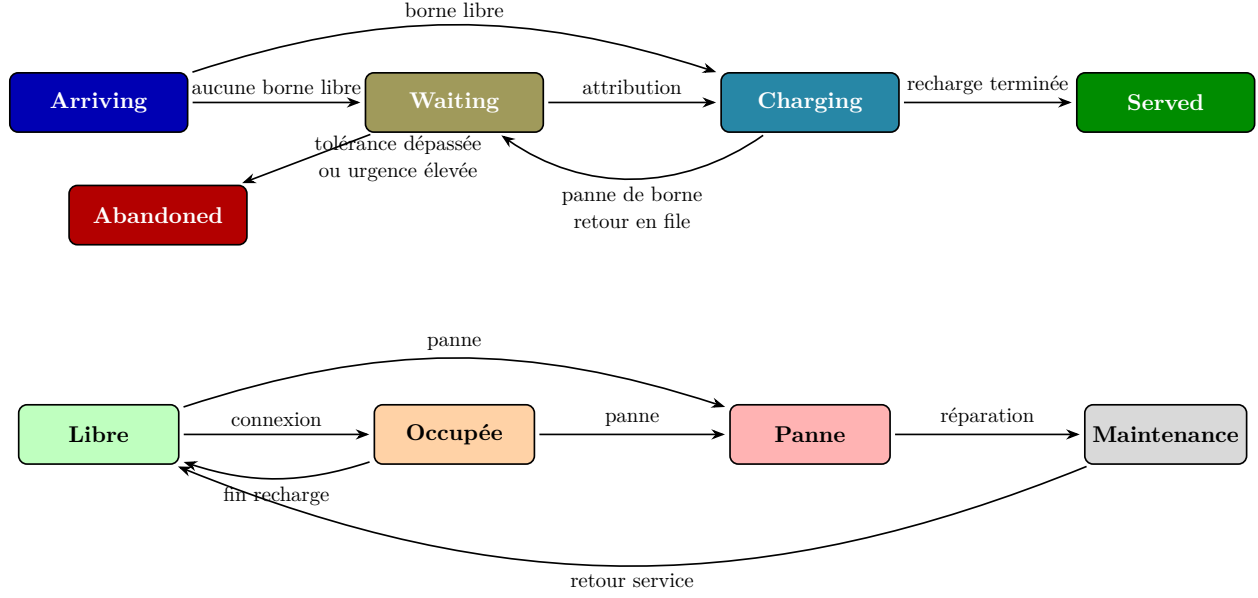


FIGURE 1 – Diagramme d'états des véhicules et des bornes de recharge.

La Figure 1 synthétise l'organisation du système. Les états du haut décrivent le parcours d'un véhicule dans la station, tandis que ceux du bas décrivent la disponibilité opérationnelle des bornes. Cette séparation permet de suivre à la fois la demande et l'état de l'offre de recharge.

3.3 File d'attente et serveurs de recharge

La station est modélisée comme une file d'attente multi-serveurs : les clients sont les véhicules et les serveurs sont les bornes. Elle est hétérogène parce que les serveurs n'ont pas tous la même puissance : les bornes lentes, accélérées et rapides ne produisent pas la même durée de recharge. Le code utilise par défaut une file unique et une politique FIFO, tout en prévoyant d'autres politiques de sélection : plus faible batterie, plus forte urgence ou respect de la préférence de puissance.

Ce formalisme est plus adapté qu'un système d'équations différentielles ordinaires, car l'ordre dans la file, la disponibilité instantanée des bornes, les pannes et les décisions d'abandon sont des événements discrets. Les équations du rapport décrivent donc les règles de calcul et les indicateurs du système plutôt qu'une dynamique continue de populations.

4 Formulation mathématique

Le modèle n'est pas formulé comme un système d'équations différentielles continu. Il s'agit d'une simulation à pas discret, avec un pas de temps de $\Delta t = 1$ minute. Les équations suivantes décrivent les calculs effectués par le code GAML à chaque cycle. Elles traduisent les mécanismes de la file d'attente et des agents : arrivée, recharge, attente, abandon et indicateurs.

4.1 Arrivée fluctuante des véhicules

Le taux instantané d'arrivée dépend de l'heure simulée $h(t)$ et augmente autour de 19 h. Le code utilise la forme :

$$\lambda(t) = \lambda_0 \left(1 + a \exp \left[-\frac{(h(t) - 19)^2}{2\sigma^2} \right] \right),$$

où λ_0 est le taux de base, a l'amplitude de variation et σ la largeur du pic. À chaque pas de temps, un véhicule peut être créé avec la probabilité :

$$p_{\text{arr}}(t) = \min \left(1, \lambda(t) \frac{\Delta t}{1 \text{ h}} \right).$$

Le modèle ajoute aussi des arrivées groupées : avec une probabilité p_{burst} , un groupe de véhicules supplémentaires est créé.

4.2 Énergie et durée de recharge

Pour un véhicule v , le déficit de batterie à combler est :

$$\Delta b_v = b_v^{\text{cible}} - b_v,$$

où b_v est le niveau courant de batterie et b_v^{cible} l'objectif de recharge. L'énergie à fournir est :

$$E_v = \frac{\Delta b_v}{100} C_v,$$

avec C_v la capacité de batterie du véhicule en kWh. La puissance réellement délivrée par une borne i est :

$$P_i^{\text{eff}} = P_i f_{\text{res}},$$

où P_i est la puissance nominale de la borne et $f_{\text{res}} \in [0.5, 1]$ représente la contrainte du réseau. La durée de recharge initialement estimée est donc :

$$T_v = \frac{E_v}{P_i^{\text{eff}}}.$$

4.3 Mise à jour de la batterie

À chaque pas, l'énergie fournie au véhicule branché est :

$$\Delta E_v = P_i^{\text{eff}} \frac{\Delta t}{1 \text{ h}}.$$

Le niveau de batterie est mis à jour par :

$$b_v(t + \Delta t) = \min \left(100, b_v(t) + 100 \frac{\Delta E_v}{C_v} \right).$$

La recharge s'arrête lorsque le véhicule atteint sa batterie cible ou lorsque le temps de recharge restant devient nul.

4.4 Estimation de l'attente et indicateurs

Lorsque l'affichage du temps d'attente est activé, la station estime l'attente d'un véhicule en file à partir de la taille de la file $Q(t)$, du nombre de bornes actives $B_{\text{act}}(t)$ et d'un temps moyen simplifié de recharge de 30 minutes :

$$\widehat{W}(t) = \frac{Q(t)}{B_{\text{act}}(t)} \times 30 \text{ minutes.}$$

Un véhicule peut abandonner si cette estimation dépasse sa tolérance ou si son temps d'attente effectif devient trop long.

Les indicateurs globaux utilisés dans le rapport sont :

$$\tau_{\text{occ}}(t) = \frac{B_{\text{occ}}(t)}{B_{\text{total}}}, \quad \bar{\tau}_{\text{occ}} = \frac{1}{n} \sum_{k=1}^n \tau_{\text{occ}}(k),$$

$$\bar{Q} = \frac{1}{n} \sum_{k=1}^n Q(k), \quad T_{\text{abandon}} = \frac{N_{\text{abandons}}}{N_{\text{arrivees}}},$$

et, pour un véhicule servi,

$$S_v = \max \left(0, 1 - \frac{W_v}{W_v^{\text{max}}} \right) \times \beta_v,$$

où W_v est le temps d'attente observé, W_v^{max} la tolérance du véhicule, et β_v vaut 1 si la préférence de puissance est respectée, sinon 0.7 dans le code.

5 Paramètres du modèle

Les paramètres retenus correspondent au code GAML mis à jour pour ce rapport. Les bornes sont réparties en trois catégories de puissance afin de représenter une station hétérogène.

TABLE 1 – Paramètres principaux de la simulation.

Catégorie	Paramètre	Valeur
Bornes	Nombre de bornes lentes	10
Bornes	Nombre de bornes accélérées	8
Bornes	Nombre de bornes rapides	5
Puissance	Borne lente	7 kW
Puissance	Borne accélérée	22 kW
Puissance	Borne rapide	50 kW
Temps	Pas de simulation	1 minute
Arrivées	Taux de base	0.5 véhicule/heure
Arrivées	Amplitude du pic	1.5
Arrivées	Largeur du pic	3 h
Stochastique	Probabilité de panne par pas	0.0005
Stochastique	Probabilité d'arrivée groupée	0.02
Stochastique	Taille d'un groupe	4 véhicules
File	Politique par défaut	FIFO
File	Organisation par défaut	File unique
Information	Affichage du temps d'attente	Activé

Chaque véhicule est créé avec un niveau de batterie initial tiré entre 5% et 40%, une batterie cible de 80%, une capacité comprise entre 30 et 80 kWh, une urgence tirée entre 0 et 1, une tolérance d'attente comprise entre 15 et 90 minutes, et une préférence de puissance choisie parmi les trois types de bornes.

6 Implémentation dans GAMA

L'implémentation est structurée autour de trois espèces. L'espèce **station** centralise les bornes, les files et l'attribution des ressources. Elle contient les actions d'enfilement, de retrait, de remise en file après panne et de sélection du prochain véhicule. La sélection peut suivre plusieurs politiques prévues dans le code : FIFO, plus faible batterie, plus forte urgence ou préférence de puissance.

L'espèce **borne** représente les points de recharge. Elle calcule la durée de recharge à partir de l'énergie demandée et de la puissance effective, met à jour la batterie du véhicule branché, libère la borne à la fin du service et gère les pannes. Lorsqu'une panne touche une borne occupée, le véhicule concerné est replacé dans la file.

L'espèce **vehicule** représente la demande. À la création, le véhicule tente d'abord une connexion directe. S'il ne trouve pas de borne disponible, il attend. Pendant l'attente, il actualise son temps d'attente, consulte l'estimation si l'information est activée et peut abandonner. Lorsqu'il est servi, il contribue au calcul du temps d'attente moyen, de la satisfaction et du nombre total de véhicules servis.

L'expérience **Station_Recharge** expose les paramètres de bornes, de puissance, de file, d'information et d'événements stochastiques. Les sorties GAMA comprennent une carte de la station, quatre séries temporelles, deux graphiques de répartition et un histogramme des indicateurs globaux.

7 Résultats de simulation et interprétation

Les figures ci-dessous proviennent des captures fournies dans le dossier **snapshots**. Elles représentent l'état de l'expérience au cycle 969. L'interprétation reste attachée aux sorties observées et aux mécanismes codés, sans extrapolation vers des scénarios qui n'ont pas été simulés ici.

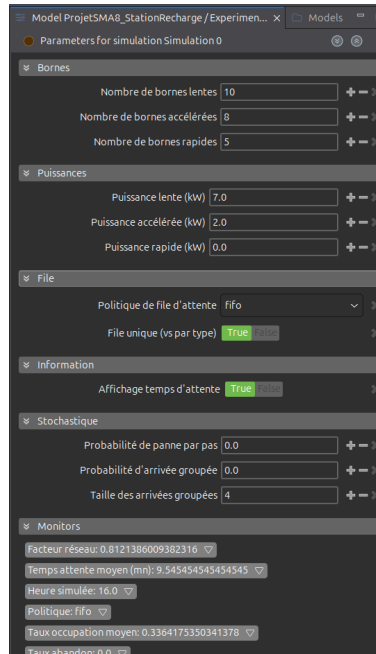


FIGURE 2 – Paramètres de lancement affichés dans l'interface GAMA. Le rapport et la copie du code associée utilisent la configuration mise à jour : 10 bornes lentes, 8 accélérées et 5 rapides.

La Figure 2 documente les paramètres utilisés au lancement de l'expérience. Cette capture est importante pour relier les résultats aux conditions de simulation : nombre de bornes, puissances, politique de file, affichage d'information et paramètres stochastiques. La configuration retenue dans le code livré avec ce rapport augmente la capacité totale de la station à 23 bornes.

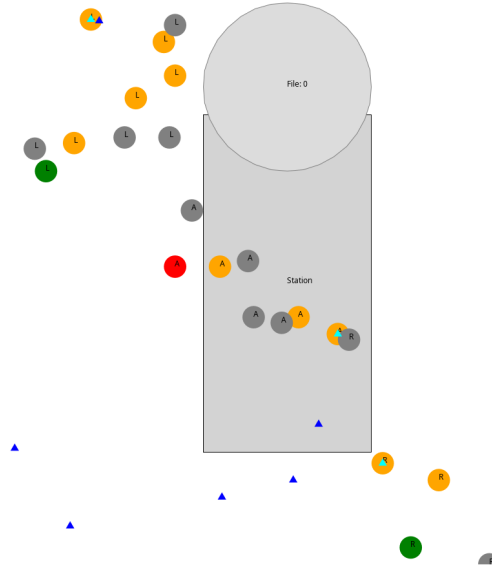


FIGURE 3 – Carte de la station au cycle 969. Les bornes et véhicules sont affichés dans l'environnement de simulation.

La Figure 3 donne une vue spatiale synthétique de la station. Les bornes sont distribuées dans l'espace et changent de couleur selon leur état opérationnel. Les véhicules en attente sont regroupés dans la zone de file, tandis que les véhicules en recharge sont associés à une borne. Cette visualisation confirme que le modèle relie les états internes des agents à une représentation spatiale lisible.

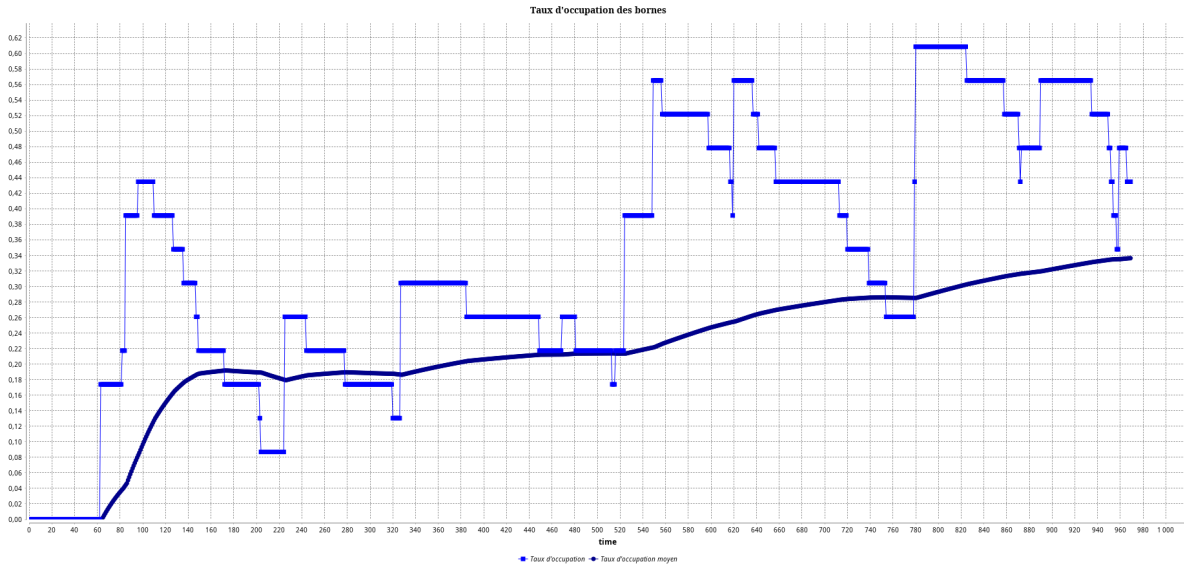


FIGURE 4 – Évolution du taux d'occupation instantané et du taux d'occupation moyen des bornes.

La Figure 4 suit la proportion de bornes occupées au cours du temps. Le taux instantané varie avec les arrivées, la durée de recharge, les libérations de bornes et les pannes éventuelles. La courbe moyenne amortit ces variations et fournit une mesure plus stable de l'utilisation globale de la station. Un taux d'occupation durablement élevé signale une forte pression sur les ressources, tandis qu'un taux faible indique une capacité disponible.

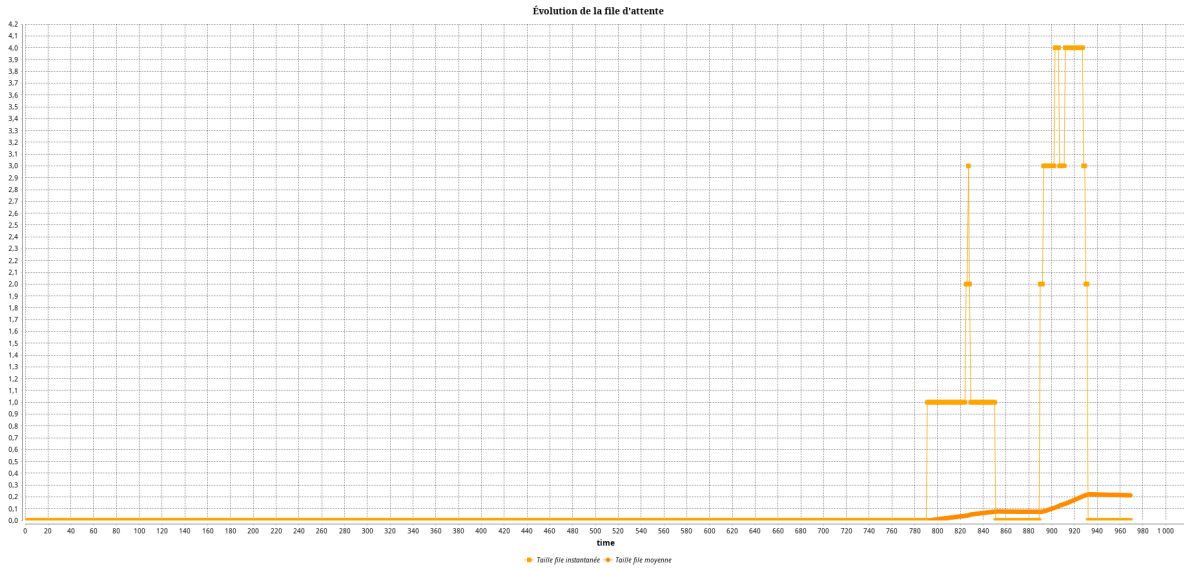


FIGURE 5 – Évolution de la taille instantanée et moyenne de la file d'attente.

La Figure 5 montre la dynamique de congestion. La file augmente lorsque le flux d'arrivée dépasse momentanément la capacité de service, notamment lors des arrivées groupées ou lorsque plusieurs bornes sont indisponibles. Elle diminue lorsque des bornes se libèrent et que la station attribue des véhicules en attente. La comparaison entre la taille instantanée et la moyenne permet de distinguer les pics ponctuels d'une congestion persistante.

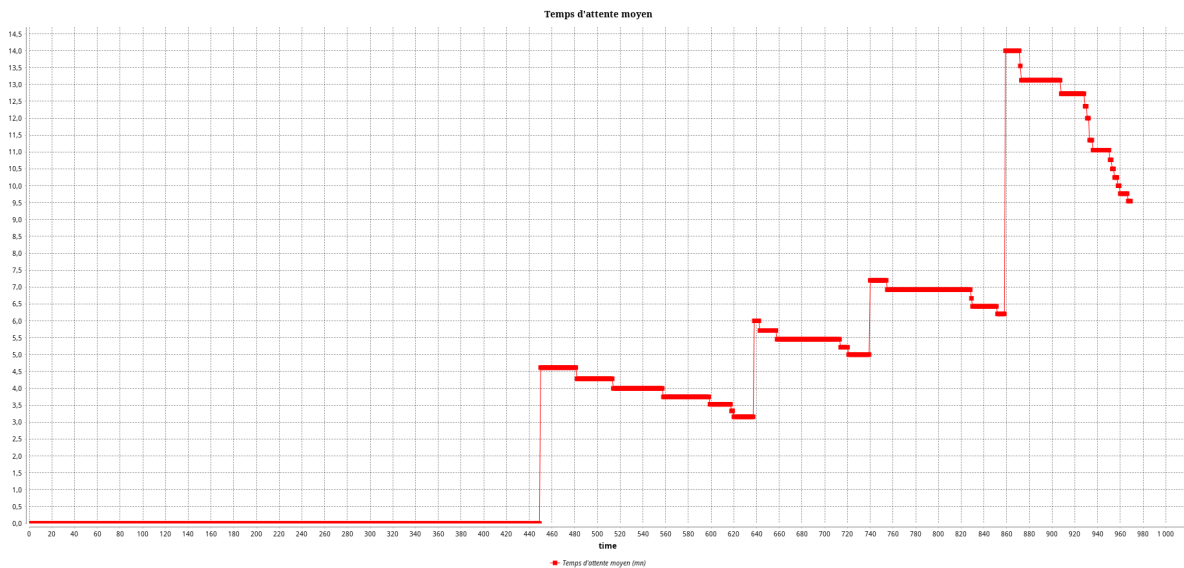


FIGURE 6 – Évolution du temps d'attente moyen des véhicules servis.

La Figure 6 présente le temps d’attente moyen calculé à partir des véhicules effectivement servis. Cet indicateur est directement lié à la satisfaction : plus l’attente se rapproche de la tolérance individuelle, plus la satisfaction diminue. Il complète la taille de file, car une file courte peut encore produire une attente importante si les recharges en cours sont longues ou si la puissance effective du réseau est réduite.

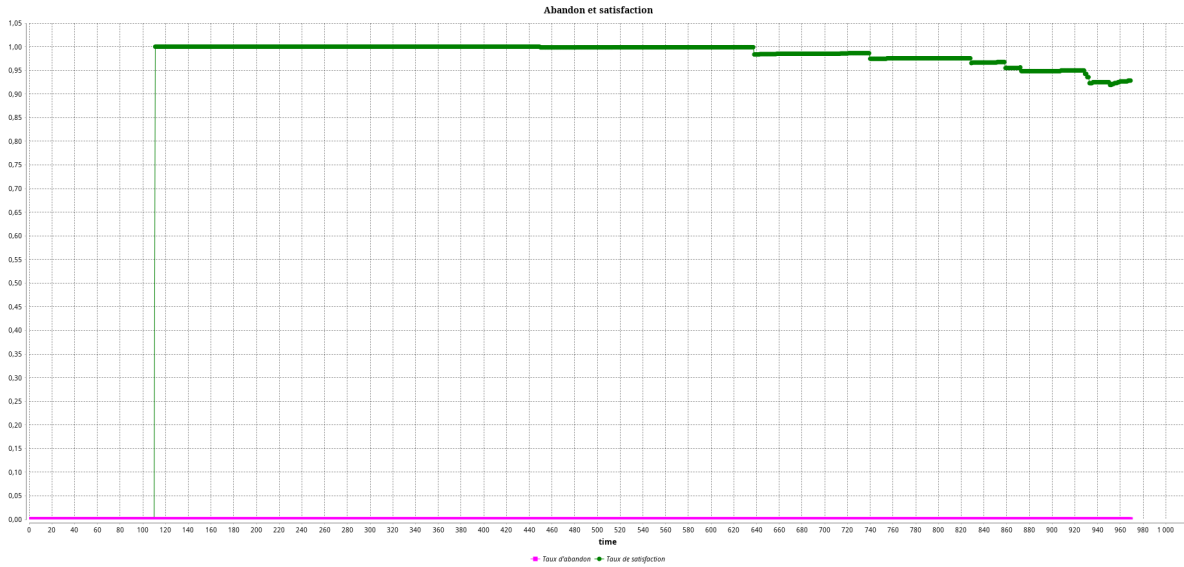


FIGURE 7 – Évolution conjointe du taux d’abandon et du taux de satisfaction.

La Figure 7 met en relation deux indicateurs complémentaires. Le taux d’abandon mesure la part des véhicules partis sans recharge, tandis que la satisfaction agrège l’effet de l’attente et du respect de la préférence de puissance. Une hausse des abandons traduit une dégradation de l’accessibilité du service. Une satisfaction élevée indique au contraire que les véhicules servis l’ont été avec une attente compatible avec leur tolérance et, autant que possible, avec leur préférence de recharge.

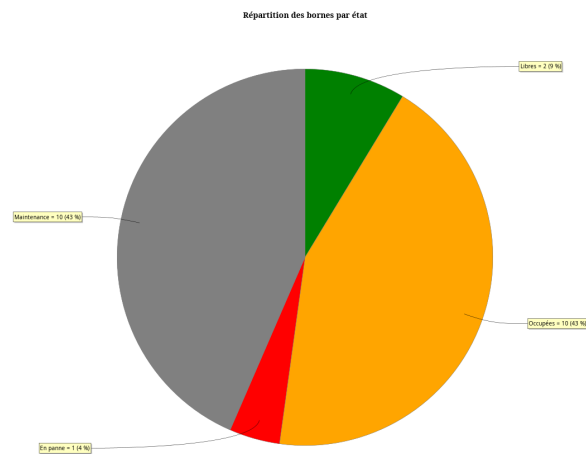


FIGURE 8 – Répartition des bornes selon leur état : libres, occupées, en panne ou en maintenance.

La Figure 8 décrit l’état opérationnel de l’offre de recharge à l’instant observé. La part des bornes

libres représente la capacité immédiatement disponible. La part occupée correspond à la charge de service en cours. Les états panne et maintenance réduisent temporairement la capacité effective, ce qui peut augmenter la file et le temps d'attente même si le nombre total de bornes installé est important.



FIGURE 9 – Répartition des véhicules par état : en attente, en recharge et servis cumulés.

La Figure 9 résume l'état de la demande. Les véhicules en attente représentent la pression immédiate sur la station. Les véhicules en recharge occupent les ressources techniques. Les véhicules servis cumulés indiquent le débit déjà réalisé par le système. Ce graphique complète la carte spatiale en donnant une lecture quantitative des états des véhicules.

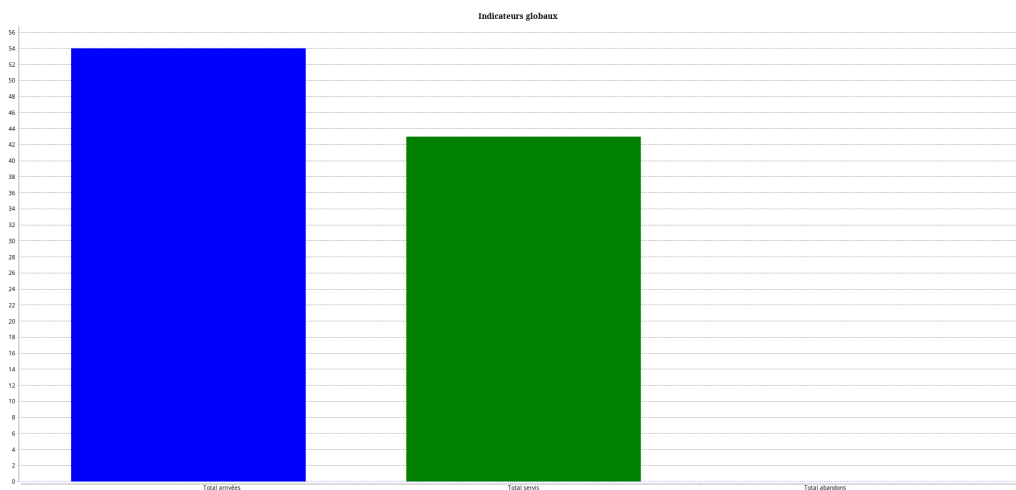


FIGURE 10 – Indicateurs globaux cumulés : arrivées totales, véhicules servis et abandons.

La Figure 10 compare les flux cumulés du système. Les arrivées totales mesurent la demande reçue par la station. Les véhicules servis représentent la demande satisfaite. Les abandons signalent les véhicules qui n'ont pas obtenu de service dans des conditions acceptables. L'écart entre arrivées et véhicules servis s'interprète donc à la lumière des véhicules encore présents dans le système et des abandons déjà enregistrés.

8 Conclusion

Ce travail a proposé une modélisation multi-agent d’une station de recharge pour véhicules électriques conformément au sujet 8. Le modèle représente explicitement les véhicules, les bornes et la station, ainsi que les principales interactions attendues : connexion directe, attente, sélection dans la file, recharge, abandon, panne et remise en file. La description par états discrets permet de suivre simultanément le parcours des véhicules et la disponibilité des bornes, tandis que la formulation mathématique précise les calculs déduits du code GAML.

La simulation met en évidence l’intérêt d’une approche multi-agent pour étudier un service soumis à une demande aléatoire et à des ressources limitées. Les indicateurs produits par GAMA permettent d’analyser l’occupation, la congestion, l’attente, l’abandon et la satisfaction. La configuration livrée dans ce rapport utilise 23 bornes, réparties en 10 lentes, 8 accélérées et 5 rapides. Cette capacité plus élevée doit réduire la pression sur la file par rapport à une station plus petite, mais la performance reste dépendante des arrivées groupées, des pannes et du facteur de puissance réseau.

La principale limite du modèle est que l’autre station mentionnée comme possibilité dans le sujet n’est pas explicitement représentée : un véhicule abandonné sort simplement du système. De plus, les préférences de puissance, l’urgence et la tolérance sont tirées aléatoirement et ne sont pas calibrées sur des données réelles. Les perspectives naturelles seraient d’exécuter plusieurs scénarios comparatifs en variant le nombre de bornes, la politique de file et l’information affichée, afin d’évaluer plus systématiquement la résilience de la station.

Références

- [1] Kasereka, S. (2025). *Système Multi-Agents*. Notes de cours, Université de Kinshasa.
- [2] Kasereka, S. (2025). *Introduction à GAMA Platform*. Support de travaux pratiques, Université de Kinshasa.
- [3] GAMA Platform. (2026). *Defining Charts*. <https://gama-platform.org/wiki/DefiningCharts>.
- [4] Farkas, C., & Telek, M. (2018). Capacity Planning of Electric Car Charging Station Based on Discrete Time Observations and MAP(2)/G/c Queue. *Periodica Polytechnica Electrical Engineering and Computer Science*, 62(3), 82–89. <https://doi.org/10.3311/PPee.11841>.
- [5] Kim, J., Son, S.-Y., Lee, J.-M., & Ha, H.-T. (2017). Scheduling and performance analysis under a stochastic model for electric vehicle charging stations. *Omega*, 66, 278–289. <https://doi.org/10.1016/j.omega.2015.11.010>.
- [6] García-Magariño, I., Palacios-Navarro, G., Lacuesta, R., & Lloret, J. (2018). ABSCEV : An agent-based simulation framework about smart transportation for reducing waiting times in charging electric vehicles. *Computer Networks*, 138, 119–135. <https://doi.org/10.1016/j.comnet.2018.03.014>.
- [7] Karlsson, J., & Grauers, A. (2023). Agent-Based Investigation of Charger Queues and Utilization of Public Chargers for Electric Long-Haul Trucks. *Energies*, 16(12), 4704. <https://doi.org/10.3390/en16124704>.
- [8] Taillandier, P., Gaudou, B., Grignard, A., Huynh, Q.-N., Marilleau, N., Caillou, P., Philippon, D., & Drogoul, A. (2019). Building, composing and experimenting complex spatial models with the GAMA platform. *GeoInformatica*, 23(2), 299–322. <https://doi.org/10.1007/s10707-018-00339-6>.

A Code GAML complet

Le code source est repris ci-dessous pour assurer la traçabilité entre la modélisation, les équations et les résultats.

```
1 /**
2  * Name: ProjetSMA8_StationRecharge
3  * Author: Duasenge Mayano
4  * Tags: SMA, GAMA, station, vehicule electrique, file d'attente
5  * Hypothèse: monde fermé - tout ce qui n'est pas déclaré est faux/inexistant.
6  */
7
8 model ProjetSMA8_StationRecharge
9
10 global {
11   /* =====
12    CONSTANTES DE TYPOLOGIE DE BORNES
13    ===== */
14   string TYPE_LENTE <- "lente";
15   string TYPE_ACC <- "acceleree";
16   string TYPE_RAPIDE <- "rapide";
17
18   /* =====
19    CONSTANTES D'ÉTATS DES BORNES
20    ===== */
21   string ETAT_LIBRE <- "libre";
22   string ETAT_OCCUPEE <- "occupee";
23   string ETAT_PANNE <- "panne";
24   string ETAT_MAINT <- "maintenance";
25
26   /* =====
27    CONSTANTES D'ÉTATS DES VÉHICULES
28    ===== */
29   string STATE_ARRIVING <- "arriving";
30   string STATE_WAITING <- "waiting";
31   string STATE_CHARGING <- "charging";
32   string STATE_SERVED <- "served";
33   string STATE_ABANDONED <- "abandoned";
34
35   /* =====
36    CONSTANTES DE POLITIQUE DE FILE D'ATTENTE
37    ===== */
38   string POL_FIFO <- "fifo";
39   string POL_LOWBAT <- "lowbat";
40   string POL_URGENCE <- "urgence";
41   string POL_PREF <- "pref";
42
43   /* =====
44    PUISSANCES DES BORNES (en kW) - PARAMÉTRABLES
45    ===== */
46   float puissance_lente <- 7.0;
47   float puissance_acc <- 22.0;
48   float puissance_rapide <- 50.0;
49
50   /* =====
51    NOMBRE DE BORNES PAR TYPE
52    ===== */
53   int nb_bornes_lentes <- 10;
54   int nb_bornes_acc <- 8;
55   int nb_bornes_rapides <- 5;
56
57   /* =====
58    CAPACITÉ ET GÉOMÉTRIE DE LA STATION
```

```

59  ===== */
60  geometry shape <- square(100);
61  float capacite_kwh_default <- 85.0; // Capacité moyenne des véhicules (kWh)
62
63  /* =====
64  PROBABILITÉS STOCHASTIQUES
65  ===== */
66  float p_panne_par_pas <- 0.0005; // Probabilité de panne à chaque pas
67  float p_fin_panne <- 0.01; // Probabilité de fin de panne
68  float p_burst <- 0.02; // Probabilité d'arrivée groupée
69  int taille_burst <- 4; // Nombre de véhicules en cas de burst
70
71  /* =====
72  PARAMÈTRES RÉSEAU
73  ===== */
74  float puissance_reseau_factor <- 1.0; // Entre 0.5 et 1.0
75
76  /* =====
77  POLITIQUE ACTIVE ET AFFICHAGE
78  ===== */
79  string politique_file <- POL_FIFO;
80  bool affichage_temps_attente <- true; // Affichage par défaut activé
81  bool file_unique <- true; // Unique file vs files par type
82
83  /* =====
84  PARAMÈTRES TEMPORELS
85  ===== */
86  float step <- 1 #mn;
87  float heure_simulee update: (cycle * step / #h) mod 24.0;
88
89  /* =====
90  PARAMÈTRES D'ARRIVÉE DES VÉHICULES (Poisson non-homogène)
91  ===== */
92  float taux_base <- 0.5; // Taux moyen (véhicules/heure)
93  float amplitude <- 1.5; // Ampleur des variations diurnes
94  float sigma <- 3.0; // Largeur de la distribution (heures)
95
96  /* =====
97  COMPTEURS KPI - INITIALISÉS À 0 (MONDE FERMÉ)
98  ===== */
99  int total_arrivees <- 0;
100  int total_servis <- 0;
101  int total_abandons <- 0;
102
103  float somme_temps_attente <- 0.0;
104  int nb_temps_attente <- 0;
105
106  float somme_satisfaction <- 0.0;
107  int nb_satisfaction <- 0;
108
109  float taux_occupation_courant <- 0.0;
110  float taille_file_courante <- 0.0;
111
112  // Accumulation pour calcul de moyennes temporelles (Comment 7, 8)
113  float somme_taux_occupation <- 0.0;
114  int nb_obs_occupation <- 0;
115  float somme_taille_file <- 0.0;
116  int nb_obs_file <- 0;
117
118  /* =====
119  VARIABLES DÉRIVÉES (mise à jour automatique via update:)
120  ===== */
121  float taux_occupation_moyen update:

```



```

122         nb_obs_occupation > 0 ? (somme_taux_occupation / nb_obs_occupation) : 0.0;
123
124     float temps_attente_moyen update:
125         nb_temps_attente > 0 ? (somme_temps_attente / nb_temps_attente) : 0.0;
126
127     float taux_abandon update:
128         total_arrivees > 0 ? (float(total_abandons) / float(total_arrivees)) : 0.0;
129
130     float taux_satisfaction update:
131         nb_satisfaction > 0 ? (somme_satisfaction / nb_satisfaction) : 0.0;
132
133     float taille_file_moyenne update:
134         nb_obs_file > 0 ? (somme_taille_file / nb_obs_file) : 0.0;
135
136     /* =====
137     INIT: CRÉATION DE LA STATION ET DES BORNES
138     ===== */
139     init {
140         // Créer 1 unique station
141         create station number: 1 {
142             location <- {50.0, 50.0}; // Centre du shape
143         }
144
145         // Créer les bornes lentes (Comment 3, 4: location et capture avec returns)
146         create borne returns: created_lentes number: nb_bornes_lentes {
147             type_borne <- TYPE_LENTE;
148             puissance_kw <- puissance_lente;
149             location <- {rnd(0, 30), rnd(0, 30)};
150         }
151         ask first(station) {
152             add all: created_lentes to: bornes;
153         }
154
155         // Créer les bornes accélérées
156         create borne returns: created_acc number: nb_bornes_acc {
157             type_borne <- TYPE_ACC;
158             puissance_kw <- puissance_acc;
159             location <- {rnd(30, 60), rnd(30, 60)};
160         }
161         ask first(station) {
162             add all: created_acc to: bornes;
163         }
164
165         // Créer les bornes rapides
166         create borne returns: created_rapides number: nb_bornes_rapides {
167             type_borne <- TYPE_RAPIDE;
168             puissance_kw <- puissance_rapide;
169             location <- {rnd(60, 100), rnd(60, 100)};
170         }
171         ask first(station) {
172             add all: created_rapides to: bornes;
173         }
174     }
175
176     /* =====
177     REFLEX GLOBAL: GÉNÉRATION D'ARRIVÉES (Poisson non-homogène)
178     ===== */
179     reflex generer_arrivees {
180         // Calcul du taux instantané : pic en soirée (19h)
181         float taux_instant <- taux_base * (1.0 + amplitude *
182             exp(-(heure_simulee - 19.0) ^ 2.0) / (2.0 * sigma ^ 2.0));
183
184         // Tirage Poisson approximé pour 1 véhicule

```

```

185         if (flip(min(1.0, taux_instant * step / #h))) {
186             create vehicule number: 1 {
187                 etat_v <- STATE_ARRIVING;
188                 t_arrivee <- time;
189             }
190             total_arrivees <- total_arrivees + 1;
191         }
192
193         // Arrivée groupée (burst) : créer taille_burst véhicules supplémentaires
194         if (flip(p_burst)) {
195             create vehicule number: taille_burst {
196                 etat_v <- STATE_ARRIVING;
197                 t_arrivee <- time;
198             }
199             total_arrivees <- total_arrivees + taille_burst;
200         }
201     }
202
203     /* =====
204     REFLEX GLOBAL: VARIATION DU FACTEUR RÉSEAU (toutes les 30 min)
205     ===== */
206     reflex variation_reseau when: (cycle mod 30) = 0 {
207         puissance_reseau_factor <- rnd(0.5, 1.0);
208     }
209
210     /* =====
211     REFLEX GLOBAL: ACCUMULATION DES KPI POUR MOYENNES TEMPORELLES
212     ===== */
213     reflex maj_kpi {
214         // Lecture des KPIs écrits par station.maj_kpi_station, accumulation pour les moyennes
temporelles
215         // (Comment 1, 7, 8: station est propriétaire unique de taux_occupation_courant et
taille_file_courante)
216         somme_taux_occupation <- somme_taux_occupation + taux_occupation_courant;
217         nb_obs_occupation <- nb_obs_occupation + 1;
218
219         somme_taille_file <- somme_taille_file + taille_file_courante;
220         nb_obs_file <- nb_obs_file + 1;
221     }
222 }
223
224 /* =====
225 ESPÈCE: STATION (1 instance, centre logistique)
226 ===== */
227 species station {
228     /* =====
229     ATTRIBUTS EXPLICITES
230     ===== */
231     list<borne> bornes <- [];
232     list<vehicule> file_unique_list <- [];
233     map<string, list<vehicule>> files_par_type <-
234         map([TYPE_LENTE::[], TYPE_ACC::[], TYPE_RAPIDE::[]]);
235     int nb_servis_local <- 0;
236     int nb_abandons_local <- 0;
237
238     /* =====
239     ACTION: ENFILER UN VÉHICULE
240     ===== */
241     action enfiler(vehicule v) {
242         if (file_unique) {
243             add v to: file_unique_list;
244         } else {
245             if (v.preference_puissance in files_par_type.keys) {

```

```

246         add v to: files_par_type[v.preference_puissance];
247     } else {
248         add v to: files_par_type[TYPE_LENTE]; // Fallback
249     }
250 }
251 }
252
253 /* =====
254 ACTION: RETIRER UN VÉHICULE DE LA FILE
255 ===== */
256 action retirer(vehicule v) {
257     remove v from: file_unique_list;
258     loop type over: files_par_type.keys {
259         remove v from: files_par_type[type];
260     }
261 }
262
263 /* =====
264 ACTION: REMETTRE EN FILE (après panne de borne)
265 ===== */
266 action requeue(vehicule v) {
267     // Retirer du lieu courant
268     do retirer(v);
269
270     // Remettre en tête de file
271     if (file_unique) {
272         add v at: 0 to: file_unique_list;
273     } else {
274         if (v.preference_puissance in files_par_type.keys) {
275             add v at: 0 to: files_par_type[v.preference_puissance];
276         } else {
277             add v at: 0 to: files_par_type[TYPE_LENTE];
278         }
279     }
280
281     // Réinitialiser l'état
282     v.etat_v <- STATE_WAITING;
283 }
284
285 /* =====
286 ACTION: SÉLECTIONNER LE PROCHAIN VÉHICULE SELON LA POLITIQUE
287 ===== */
288 vehicule selectionner_suivant(string type_libere) {
289     list<vehicule> file_active <- [];
290
291     // Récupérer la file appropriée
292     if (file_unique) {
293         file_active <- file_unique_list;
294     } else {
295         // Chercher d'abord dans la file du type libéré
296         if (type_libere in files_par_type.keys and !empty(files_par_type[type_libere])) {
297             file_active <- files_par_type[type_libere];
298         } else {
299             // Sinon, combiner toutes les files
300             loop type over: files_par_type.keys {
301                 add all: files_par_type[type] to: file_active;
302             }
303         }
304     }
305
306     if (empty(file_active)) {
307         return nil;
308     }

```

```

309
310     vehicule selection <- nil;
311
312     // Appliquer la politique (Comment 1: switch...match au lieu de switch...case)
313     switch (politique_file) {
314         match POL_FIFO {
315             selection <- first(file_active);
316         }
317         match POL_LOWBAT {
318             vehicule v_lowbat <- file_active[0];
319             loop v over: file_active {
320                 if (v.batterie < v_lowbat.batterie) {
321                     v_lowbat <- v;
322                 }
323             }
324             selection <- v_lowbat;
325         }
326         match POL_URGENCE {
327             vehicule v_urgent <- file_active[0];
328             loop v over: file_active {
329                 if (v.urgence > v_urgent.urgence) {
330                     v_urgent <- v;
331                 }
332             }
333             selection <- v_urgent;
334         }
335         match POL_PREF {
336             // Chercher d'abord avec préférence
337             list<vehicule> avec_pref <- file_active where (each.preference_puissance =
type_libere);
338             if (!empty(avec_pref)) {
339                 selection <- first(avec_pref);
340             } else {
341                 selection <- first(file_active);
342             }
343         }
344         default {
345             selection <- first(file_active);
346         }
347     }
348
349     return selection;
350 }
351
352 /* =====
353 REFLEX: ATTRIBUER LES BORNES LIBRES AUX VÉHICULES EN FILE
354 ===== */
355 reflex attribuer_bornes_libres {
356     // Parcourir les bornes libres
357     list<borne> bornes_libres <- bornes where (each.etat = ETAT_LIBRE);
358
359     loop b over: bornes_libres {
360         // Sélectionner le prochain véhicule
361         vehicule v <- selectionner_suivant(b.type_borne);
362
363         if (v != nil) {
364             // Retirer de la file
365             do retirer(v);
366
367             // Démarrer la recharge
368             ask b {
369                 do demarrer_recharge(v);
370             }

```

```

371         v.etat_v <- STATE_CHARGING;
372         v.borne_assignee <- b;
373     }
374 }
375 }
376 }
377
378 /* =====
379 ACTION: ESTIMER LE TEMPS D'ATTENTE
380 ===== */
381 float estimer_attente(vehicule v) {
382     // Calcul du nombre de bornes compatibles (Comment 9: fix copy-paste bug)
383     int nb_bornes_compatibles <- file_unique ?
384         length(bornes where (each.etat = ETAT_LIBRE)) :
385         length(bornes where (each.type_borne = v.preference_puissance and each.etat =
ETAT_LIBRE));
386     int nb_bornes_actives <- max(1, nb_bornes_compatibles + 1);
387
388     // Taille de la file
389     int taille_file <- 0;
390     if (file_unique) {
391         taille_file <- length(file_unique_list);
392     } else {
393         loop t over: files_par_type.keys {
394             taille_file <- taille_file + length(files_par_type[t]);
395         }
396     }
397
398     // Temps moyen de recharge estimé (simplifié)
399     float temps_recharge_moyen <- 30.0 * #mn; // 30 minutes en moyenne
400
401     // Estimation : (taille_file / bornes_actives) × temps_moyen
402     float estimation <- (float(taille_file) / float(nb_bornes_actives)) * temps_recharge_moyen;
403
404     return estimation;
405 }
406
407 /* =====
408 REFLEX: MISE À JOUR DES KPI DE LA STATION
409 ===== */
410 reflex maj_kpi_station {
411     // Taux d'occupation
412     int nb_bornes_total <- length(bornes);
413     int nb_bornes_occupees <- length(bornes where (each.etat = ETAT_OCCUPEE));
414
415     if (nb_bornes_total > 0) {
416         taux_occupation_courant <- float(nb_bornes_occupees) / float(nb_bornes_total);
417     } else {
418         taux_occupation_courant <- 0.0;
419     }
420
421     // Taille de la file
422     if (file_unique) {
423         taille_file_courante <- float(length(file_unique_list));
424     } else {
425         int total_en_file <- 0;
426         loop type over: files_par_type.keys {
427             total_en_file <- total_en_file + length(files_par_type[type]);
428         }
429         taille_file_courante <- float(total_en_file);
430     }
431 }
432

```

```

433 /* =====
434 ASPECT VISUEL
435 ===== */
436 aspect base {
437     // Rectangle représentant la station (borne grisâtre)
438     draw rectangle(30.0, 60.0) at: {50.0, 50.0} color: #lightgray border: #black;
439
440     // Zone de file d'attente (cercle gris pâle)
441     draw circle(15.0) at: {50.0, 15.0} color: #gainsboro border: #gray;
442
443     // Texte d'information
444     draw text: "Station" color: #black size: 2.0 at: {50.0, 50.0};
445     draw text: ("File: " + string(int(taille_file_courante)))
446         color: #black size: 1.5 at: {50.0, 15.0};
447 }
448 }
449
450 /* =====
451 ESPÈCE: BORNE (points de recharge)
452 ===== */
453 species borne {
454     /* =====
455     ATTRIBUTS EXPLICITES (MONDE FERMÉ)
456     ===== */
457     string type_borne <- TYPE_LENTE; // Type de borne (à modifier à la création)
458     float puissance_kw <- puissance_lente; // Puissance en kW (cohérent avec type_borne)
459     string etat <- ETAT_LIBRE; // État courant (par défaut explicite)
460     vehicule vehicule_branche <- nil; // Véhicule en cours de recharge
461     float temps_panne_restant <- 0.0; // Durée restante de panne (en mn)
462     int nb_recharges_effectuees <- 0; // Compteur de recharges complétées
463     int nb_pas_occupee <- 0; // Nombre de pas où la borne était occupée
464     int nb_pas_total <- 0; // Nombre total de pas de simulation
465
466     /* =====
467     ACTION: DEMARRER UNE RECHARGE
468     ===== */
469     action demarrer_recharge(vehicule v) {
470         etat <- ETAT_OCCUPEE;
471         vehicule_branche <- v;
472
473         // Calcul du temps de recharge en heures (Comment 6: utiliser v.capacite_kwh)
474         float delta_batterie <- v.batterie_cible - v.batterie; // % à charger
475         float energie_kwh <- (delta_batterie / 100.0) * v.capacite_kwh; // kWh à fournir
476         float puissance_effective <- puissance_kw * puissance_reseau_factor; // kW effectifs
477
478         if (puissance_effective > 0.0) {
479             v.temps_recharge_restant <- (energie_kwh / puissance_effective) * #h; // en unités
480             } else {
481                 v.temps_recharge_restant <- 1.0 * #h; // Fallback sûr
482             }
483         }
484
485     /* =====
486     ACTION: LIBÉRER LA BORNE
487     ===== */
488     action liberer {
489         etat <- ETAT_LIBRE;
490         vehicule_branche <- nil;
491         nb_recharges_effectuees <- nb_recharges_effectuees + 1;
492     }
493
494     /* =====

```

```

495     REFLEX: RECHARGER LE VÉHICULE BRANCHÉ
496     ===== */
497     reflex recharger when: (etat = ETAT_OCCUPEE and vehicule_branche != nil) {
498         // Décrémenter le temps restant (pondéré par le facteur réseau)
499         float decrement <- step * puissance_reseau_factor;
500         vehicule_branche.temps_recharge_restant <- vehicule_branche.temps_recharge_restant - decrement;
501
502         // Mise à jour du niveau de batterie proportionnellement (Comment 6: utiliser v.capacite_kwh)
503         float puissance_effective <- puissance_kw * puissance_reseau_factor;
504         if (puissance_effective > 0.0 and vehicule_branche.capacite_kwh > 0.0) {
505             float kwh_fournis <- (puissance_effective * step / #h);
506             float pct_batterie <- (kwh_fournis / vehicule_branche.capacite_kwh) * 100.0;
507             vehicule_branche.batterie <- min(100.0, vehicule_branche.batterie + pct_batterie);
508         }
509
510         // Fin de recharge : batterie cible atteinte
511         if (vehicule_branche.batterie >= vehicule_branche.batterie_cible
512             or vehicule_branche.temps_recharge_restant <= 0.0) {
513             vehicule_branche.etat_v <- STATE_SERVED;
514
515             // Libération de la borne (Comment 11: do liberer; au lieu de liberer();)
516             do liberer;
517         }
518     }
519
520     /* =====
521     REFLEX: TOMBER EN PANNE
522     ===== */
523     reflex tomber_en_panne when: (etat = ETAT_LIBRE or etat = ETAT_OCCUPEE) and flip(p_panne_par_pas) {
524         // Si occupée, le véhicule branché rejoint la file (Comment 5: re-queue le véhicule)
525         if (etat = ETAT_OCCUPEE and vehicule_branche != nil) {
526             vehicule v <- vehicule_branche;
527             station s <- first(station);
528             if (s != nil) {
529                 ask s {
530                     do requeue(v);
531                 }
532             }
533             vehicule_branche <- nil;
534         }
535
536         // Passage en panne avec durée aléatoire
537         etat <- ETAT_PANNE;
538         temps_panne_restant <- rnd(30.0, 180.0) * #mn; // Entre 30 et 180 minutes
539     }
540
541     /* =====
542     REFLEX: RÉPARER LA BORNE
543     ===== */
544     reflex reparer when: etat = ETAT_PANNE {
545         // Décrémenter de la durée de panne
546         temps_panne_restant <- temps_panne_restant - step;
547
548         if (temps_panne_restant <= 0.0) {
549             // Passage en maintenance temporaire
550             etat <- ETAT_MAINT;
551             temps_panne_restant <- rnd(5.0, 15.0) * #mn; // Maintenance 5-15 min
552         }
553     }
554
555     /* =====
556     REFLEX: COMPTER L'OCCUPATION
557     ===== */

```

```

558     reflex compteur_occupation {
559         nb_pas_total <- nb_pas_total + 1;
560         if (etat = ETAT_OCCUPEE) {
561             nb_pas_occupee <- nb_pas_occupee + 1;
562         }
563     }
564
565     /* =====
566     ASPECT VISUEL
567     ===== */
568     aspect base {
569         // Couleur selon l'état
570         rgb couleur <- etat = ETAT_LIBRE ? #green : (etat = ETAT_OCCUPEE ? #orange : (etat = ETAT_PANNE
? #red : (etat = ETAT_MAINT ? #gray : #white)));
571
572         // Label abrégé du type
573         string label <- type_borne = TYPE_LENTE ? "L" : (type_borne = TYPE_ACC ? "A" : (type_borne =
TYPE_RAPIDE ? "R" : "?"));
574
575         // Cercle représentant la borne
576         draw circle(2.0) color: couleur;
577         // Texte au-dessus indiquant le type
578         draw text: label color: #black size: 1.0;
579     }
580 }
581
582 /* =====
583 ESPÈCE: VEHICULE (agents dynamiques)
584 ===== */
585 species vehicule {
586     /* =====
587     ATTRIBUTS EXPLICITES (MONDE FERMÉ)
588     ===== */
589     float batterie <- rnd(5.0, 40.0);           // % initial de batterie
590     float batterie_cible <- 80.0;               // Objectif de recharge
591     float capacite_kwh <- rnd(30.0, 80.0);       // Capacité du réservoir
592     float urgence <- rnd(0.0, 1.0);             // Niveau d'urgence (0=aucune, 1=max)
593     float tolerance_attente <- rnd(15.0, 90.0) * #mn; // Tolérance d'attente maximale
594     string preference_puissance <- one_of([TYPE_LENTE, TYPE_ACC, TYPE_RAPIDE]); // Type préféré
595     string etat_v <- STATE_ARRIVING;           // État du véhicule
596     float t_arrivee <- time;                   // Temps d'arrivée initial
597     float temps_attente_actuel <- 0.0;          // Durée d'attente accumulée
598     float temps_recharge_restant <- 0.0;        // Temps restant pour recharge
599     borne borne_assignee <- nil;                // Borne assignée (si recharge)
600     float seuil_urgence_quitter <- 0.85;        // Seuil d'urgence pour abandon
601
602     /* =====
603     ACTION: ESSAYER UNE CONNEXION À UNE BORNE
604     ===== */
605     action essayer_connexion {
606         // Chercher une borne libre correspondant à la préférence
607         list<borne> bornes_libres <- (borne where (each.etat = ETAT_LIBRE));
608
609         if (empty(bornes_libres)) {
610             // Aucune borne libre : rejoindre la file
611             etat_v <- STATE_WAITING;
612             station first_station <- first(station);
613             if (first_station != nil) {
614                 ask first_station {
615                     do enfiler(myself);
616                 }
617             }
618         } else {

```



```

619         // Chercher d'abord une borne avec la puissance préférée
620         list<borne> bornes_pref <- bornes_libres where (each.type_borne = preference_puissance)
;
621
622         if (!empty(bornes_pref)) {
623             borne_assignee <- first(bornes_pref);
624         } else {
625             borne_assignee <- first(bornes_libres);
626         }
627
628         // Démarrer la recharge
629         if (borne_assignee != nil) {
630             ask borne_assignee {
631                 do demarrer_recharge(myself);
632             }
633             etat_v <- STATE_CHARGING;
634         }
635     }
636 }
637
638 /* =====
639 ACTION D'INITIALISATION: APPELÉE À LA CRÉATION
640 ===== */
641 init {
642     do essayer_connexion;
643 }
644
645 /* =====
646 REFLEX: GESTION DE L'ATTENTE EN FILE
647 ===== */
648 // Réévaluation périodique toutes les 5 minutes (Comment 12, 2)
649 reflex attente when: etat_v = STATE_WAITING {
650     // Accumulation du temps d'attente
651     temps_attente_actuel <- temps_attente_actuel + step;
652
653     // Re-query ETA every 5 simulated minutes; modulo detects boundary crossing
654     if (affichage_temps_attente and (int(temps_attente_actuel) mod (5 * 60)) < int(step)) {
655         float eta_estime <- 0.0;
656         station first_station <- first(station);
657         if (first_station != nil) {
658             ask first_station {
659                 eta_estime <- estimer_attente(myself);
660             }
661         }
662
663         // Abandon préemptif si ETA trop long
664         if (eta_estime > tolerance_attente) {
665             etat_v <- STATE_ABANDONED;
666         }
667     }
668
669     // Abandon par dépassement de tolérance
670     if (temps_attente_actuel > tolerance_attente) {
671         etat_v <- STATE_ABANDONED;
672     }
673
674     // Abandon par urgence élevée + attente prolongée
675     if (urgence > seuil_urgence_quitter and
676         temps_attente_actuel > (tolerance_attente / 2.0)) {
677         etat_v <- STATE_ABANDONED;
678     }
679
680     // Traitement de l'abandon : retrait de la file et statistiques

```

```

681         if (etat_v = STATE_ABANDONED) {
682             station first_station <- first(station);
683             if (first_station != nil) {
684                 ask first_station {
685                     do retirer(myself);
686                 }
687             }
688             total_abandons <- total_abandons + 1;
689             somme_satisfaction <- somme_satisfaction + 0.0;
690             nb_satisfaction <- nb_satisfaction + 1;
691             do die;
692         }
693     }
694
695     /* =====
696     REFLEX: COMPTABILISER LES VEHICULES SERVIS
697     ===== */
698     reflex comptabiliser_servi when: etat_v = STATE_SERVED {
699         // Accumulation des temps d'attente
700         somme_temps_attente <- somme_temps_attente + temps_attente_actuel;
701         nb_temps_attente <- nb_temps_attente + 1;
702
703         // Calcul de la satisfaction
704         float satisfaction_temps <- max(0.0, 1.0 - (temps_attente_actuel / tolerance_attente));
705         float bonus_preference <- (borne_assignee != nil and
706                                     preference_puissance = borne_assignee.type_borne) ? 1.0 : 0.7;
707         float satisfaction <- satisfaction_temps * bonus_preference;
708
709         // Enregistrement de la satisfaction
710         somme_satisfaction <- somme_satisfaction + satisfaction;
711         nb_satisfaction <- nb_satisfaction + 1;
712
713         // Incrémentation des véhicules servis
714         total_servis <- total_servis + 1;
715
716         // Destruction du véhicule
717         do die;
718     }
719
720     /* =====
721     ASPECT VISUEL
722     ===== */
723     aspect base {
724         // Couleur selon l'état du véhicule
725         rgb couleur <- etat_v = STATE_ARRIVING ? #blue : (etat_v = STATE_WAITING ? #yellow : (etat_v =
STATE_CHARGING ? #cyan : (etat_v = STATE_SERVED ? #green : (etat_v = STATE_ABANDONED ? #red : #white)))));
726
727         // Positionnement : près de la borne si en charge, sinon dans la zone file
728         point pos <- location;
729         if (etat_v = STATE_CHARGING and borne_assignee != nil) {
730             pos <- borne_assignee.location; // Comment 3: location au lieu de position
731         } else if (etat_v = STATE_WAITING) {
732             // Placer dans une zone "file" (zone de waiting)
733             pos <- {50.0 + rnd(-10.0, 10.0), 10.0 + rnd(-5.0, 5.0)};
734         }
735
736         // Triangle représentant le véhicule
737         draw triangle(1.5) at: pos color: couleur;
738     }
739 }
740
741 /* =====
742 EXPERIMENT: INTERFACE GRAPHIQUE

```

```

743  ===== */
744  experiment Station_Recharge type: gui {
745      /* =====
746          PARAMÈTRES EXPOSÉS À L'INTERFACE UTILISATEUR
747          ===== */
748      parameter "Nombre de bornes lentes" var: nb_bornes_lentes category: "Bornes";
749      parameter "Nombre de bornes accélérées" var: nb_bornes_acc category: "Bornes";
750      parameter "Nombre de bornes rapides" var: nb_bornes_rapides category: "Bornes";
751
752      parameter "Puissance lente (kW)" var: puissance_lente category: "Puissances";
753      parameter "Puissance accélérée (kW)" var: puissance_acc category: "Puissances";
754      parameter "Puissance rapide (kW)" var: puissance_rapide category: "Puissances";
755
756      parameter "Politique de file d'attente" var: politique_file
757          among: [POL_FIFO, POL_LOWBAT, POL_URGENCE, POL_PREF] category: "File";
758      parameter "File unique (vs par type)" var: file_unique category: "File";
759
760      parameter "Affichage temps d'attente" var: affichage_temps_attente category: "Information";
761
762      parameter "Probabilité de panne par pas" var: p_panne_par_pas category: "Stochastique";
763      parameter "Probabilité d'arrivée groupée" var: p_burst category: "Stochastique";
764      parameter "Taille des arrivées groupées" var: taille_burst category: "Stochastique";
765
766      /* =====
767          SORTIES GRAPHIQUES ET MONITEURS
768          ===== */
769      output {
770          // Affichage de la carte de la station
771          display CarteStation type: 2d refresh: every(1#cycle) {
772              species station aspect: base;
773              species borne aspect: base;
774              species vehicule aspect: base;
775          }
776
777          // Série temporelle du taux d'occupation
778          display Occupation_Series refresh: every(1#cycle) type: 2d {
779              chart "Taux d'occupation des bornes" type: series {
780                  data "Taux d'occupation" value: taux_occupation_courant color: rgb('blue');
781                  data "Taux d'occupation moyen" value: taux_occupation_moyen color: rgb('
darkblue');
782              }
783          }
784
785          // Série temporelle de la taille de file (Comment 8: ajouter taille_file_moyenne)
786          display File_Series refresh: every(1#cycle) type: 2d {
787              chart "Évolution de la file d'attente" type: series {
788                  data "Taille file instantanée" value: taille_file_courante color: rgb('orange');
789                  data "Taille file moyenne" value: taille_file_moyenne color: rgb('darkorange');
790              }
791          }
792
793          // Série temporelle du temps d'attente moyen
794          display Attente_Series refresh: every(1#cycle) type: 2d {
795              chart "Temps d'attente moyen" type: series {
796                  data "Temps d'attente moyen (mn)" value: temps_attente_moyen color: rgb('red');
797              }
798          }
799
800          // Série temporelle des taux d'abandon et satisfaction
801          display Abandon_Satisfaction refresh: every(1#cycle) type: 2d {
802              chart "Abandon et satisfaction" type: series {
803                  data "Taux d'abandon" value: taux_abandon color: rgb('magenta');

```

```

804         data "Taux de satisfaction" value: taux_satisfaction color: rgb('green');
805     }
806 }
807
808 // Répartition des états des bornes (camembert)
809 display Repartition_Bornes_Pie refresh: every(5#cycle) {
810     chart "Répartition des bornes par état" type: pie {
811         data "Libres" value: borne count (each.etat = ETAT_LIBRE) color: rgb('green');
812         data "Occupées" value: borne count (each.etat = ETAT_OCCUPEE) color: rgb('
orange');
813         data "En panne" value: borne count (each.etat = ETAT_PANNE) color: rgb('red');
814         data "Maintenance" value: borne count (each.etat = ETAT_MAINT) color: rgb('gray
');
815     }
816 }
817
818 // Répartition des véhicules par état (histogramme) (Comment 14: remplacer Arriving par Served)
819 display Repartition_Vehicules_Hist refresh: every(5#cycle) {
820     chart "Répartition des véhicules par état" type: histogram {
821         data "Waiting" value: vehicule count (each.etat_v = STATE_WAITING) color: rgb('
yellow');
822         data "Charging" value: vehicule count (each.etat_v = STATE_CHARGING) color: rgb
('cyan');
823         data "Served (cumul)" value: total_servis color: rgb('green');
824     }
825 }
826
827 // Indicateurs globaux (histogramme)
828 display Indicateurs_Globaux_Hist refresh: every(10#cycle) {
829     chart "Indicateurs globaux" type: histogram {
830         data "Total arrivées" value: total_arrivees color: rgb('blue');
831         data "Total servis" value: total_servis color: rgb('green');
832         data "Total abandons" value: total_abandons color: rgb('red');
833     }
834 }
835
836 // Moniteurs d'indicateurs clés
837 monitor "Heure simulée" value: heure_simulee;
838 monitor "Politique" value: politique_file;
839 monitor "Facteur réseau" value: puissance_reseau_factor;
840 monitor "Taux occupation moyen" value: taux_occupation_moyen;
841 monitor "Temps attente moyen (mn)" value: temps_attente_moyen;
842 monitor "Taux abandon" value: taux_abandon;
843 monitor "Taux satisfaction" value: taux_satisfaction;
844 }
845 }

```